

UNIVERSIDAD EL BOSQUE

Departamento de Matemáticas

Programa: Matemáticas y Ciencia de Datos

Análisis de riesgo de incumplimiento con un modelo de regresión Logit

Módulo 1, Unidad 1 — Guía de Aprendizaje 2
Actividades 3 y 4

Presentado por

Sergio Preciado

Curso

Inteligencia Artificial para Asuntos Financieros

Docente

William Arley Rincón Gómez

Bogotá D.C., 2025

1 ¿Qué vamos a hacer y por qué?

Queremos responder una pregunta de un banco: **¿este cliente va a pagar su crédito o no?** La respuesta solo tiene dos opciones (sí o no), y para ese tipo de preguntas se usa el **modelo logit**.

La idea es sencilla. El modelo toma los datos del cliente (edad, ingresos, score, etc.) y devuelve **una probabilidad entre 0 y 1**. Por ejemplo, 0,85 significa “hay 85 % de posibilidades de que este cliente pague”.

Para leer los resultados vamos a usar el **OR** (*odds ratio*), que es simplemente **un multiplicador de las posibilidades**:

Si el OR vale...	Significa que la variable...
más de 1 (ej. 2,0)	aumenta las posibilidades (aquí, las duplica)
menos de 1 (ej. 0,5)	reduce las posibilidades (aquí, a la mitad)
exactamente 1	no cambia nada

Este trabajo tiene dos partes, tal como las pide la guía:

1. **Parte 1:** inventar (simular) 1.000 clientes y aplicarles el modelo.
2. **Parte 2:** usar la base real **Traindata** del curso.

2 Parte 1: simulación de 1.000 clientes

2.1 Paso 1: cargar las librerías

```
# ggplot2 sirve para hacer gráficos bonitos.
# readxl sirve para abrir archivos de Excel (lo usaremos en la Parte 2).
library(ggplot2)
library(readxl)
```

2.2 Paso 2: inventar los datos de 1.000 clientes

```
# set.seed() hace que los "números al azar" sean siempre los mismos.
# Así, si otra persona corre este código, obtiene exactamente mis resultados.
set.seed(123)

n <- 1000 # vamos a inventar 1.000 clientes

# rnorm(cuántos, promedio, variación) inventa números alrededor de un promedio.
# rbinom() inventa ceros y unos.
# Las variables se crean en el mismo orden en que las lista la guía.
edad <- rnorm(n, 40, 10) # edades alrededor de 40 años
ingresos <- rnorm(n, 3000, 1000) # ingresos alrededor de 3.000
score <- rnorm(n, 600, 50) # score alrededor de 600 puntos
genero <- rbinom(n, 1, 0.5) # 0 = mujer, 1 = hombre, mitad y mitad
monto_prestamo <- rnorm(n, 5000, 1500) # préstamos alrededor de 5.000
```

2.3 Paso 3: calcular la probabilidad de que cada cliente incumpla

```
# Esta cuenta larga es la fórmula que da la guía. Cada número dice cuánto
# influye cada variable. Todavía no es una probabilidad, es un puntaje interno.
lin_pred <- -3 - 0.03 * edad - 0.0004 * ingresos + 0.01 * score +
            0.8 * genero + 0.0001 * monto_prestamo
```

```
# Esta línea convierte ese puntaje en una probabilidad entre 0 y 1.
prob <- 1 / (1 + exp(-lin_pred))
```

```
# Miramos cómo quedaron las probabilidades:
summary(prob)
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>  0.302   0.706   0.815   0.782   0.881   0.986
```

2.4 Paso 4: decidir quién incumple y quién no

```
# rbinom() lanza una "moneda cargada" con la probabilidad de cada cliente.
# El resultado es la variable que queremos explicar: 0 = paga, 1 = no paga.
default <- rbinom(n, 1, prob)

# Juntamos todo en una tabla (data.frame) para trabajar ordenados.
datos1 <- data.frame(default, edad, ingresos, score, genero, monto_prestamo)

# ¿Cuántos incumplieron?
table(default)
```

```
#> default
#>      0      1
#> 215 785
```

De los 1.000 clientes inventados, **785 no pagaron**, o sea el **78.5 %**. Este número nos va a servir más adelante.

2.5 Paso 5: ajustar el modelo logit

```
# glm() es la función que ajusta el modelo. Se lee así:
#   default ~ edad + ingresos + ... -> "explica default usando estas variables"
#   family = binomial(link = "logit") -> "usa el modelo logit"
#   data = datos1 -> "los datos están en esta tabla"
modelo1 <- glm(default ~ edad + ingresos + score + genero + monto_prestamo,
               family = binomial(link = "logit"),
               data = datos1)
```

2.6 Paso 6: ver la salida del modelo

```
summary(modelo1)
```

```
#>
#> Call:
#> glm(formula = default ~ edad + ingresos + score + genero + monto_prestamo,
#>      family = binomial(link = "logit"), data = datos1)
#>
#> Coefficients:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  -0.9783100  1.1208247  -0.87   0.38275
#> edad         -0.0342015  0.0083046  -4.12 0.000038154 ***
#> ingresos     -0.0003151  0.0000822  -3.83   0.00013 ***
#> score         0.0068442  0.0016859   4.06 0.000049123 ***
#> genero        0.9407444  0.1688650   5.57 0.000000025 ***
#> monto_prestamo 0.0000411  0.0000542   0.76   0.44840
```

```
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#> Null deviance: 1041.01  on 999  degrees of freedom
#> Residual deviance: 955.06  on 994  degrees of freedom
#> AIC: 967.1
#>
#> Number of Fisher Scoring iterations: 4
```

2.7 Paso 7: interpretar los resultados

Para leer esto más fácil, convertimos los coeficientes en OR (multiplicadores):

```
# exp() convierte el coeficiente en el multiplicador (OR).
round(exp(coef(modelo1)), 4)
```

```
#>      (Intercept)          edad      ingresos      score      genero
#>      0.3759      0.9664      0.9997      1.0069      2.5619
#> monto_prestamo
#>      1.0000
```

Tabla 2: Parte 1. Resultados del modelo, traducidos a lenguaje sencillo.

Si aumenta...	OR	Efecto sobre el riesgo de no pagar	¿Es confiable?
Edad (1 año más)	0.966	Baja el riesgo	Sí
Ingresos (100 más)	0.969	Baja el riesgo	Sí
Score (10 puntos más)	1.071	Sube el riesgo	Sí
Ser hombre	2.562	Sube el riesgo	Sí
Préstamo (1.000 más)	1.042	Sube el riesgo	No

En palabras simples:

- **Edad:** los clientes mayores incumplen menos. Cada año que pasa baja un poco el riesgo (OR = 0.966, o sea un 3.4 % menos).
- **Ingresos:** quien gana más, paga mejor. Es lo esperado.
- **Ser hombre:** en estos datos los hombres incumplen más. El OR de 2.56 quiere decir que sus posibilidades de no pagar son unas 2.6 veces las de las mujeres.
- **Score:** aquí aparece algo raro, y hay que decirlo: el modelo dice que **un mejor score sube el riesgo**. Eso no tiene sentido en la vida real, porque un score alto es justamente señal de buen pagador. La explicación es que la fórmula del Paso 3 (la que da la guía) trae ese signo así, y el modelo simplemente repite lo que está en los datos inventados.
- **Monto del préstamo:** sube el riesgo, pero muy poquito, y la columna “¿Es confiable?” dice **No**: con estos datos no podemos asegurar que realmente influya.

Una nota importante: el 78.5 % de incumplimiento de estos datos inventados es altísimo. Ningún banco real tiene tantos clientes que no pagan. Pasa por lo mismo del score: la fórmula de la guía infla el riesgo. El modelo está bien hecho; **los datos inventados son los que no son realistas**.

2.8 Paso 8: un gráfico para ver el resultado

```
# Guardamos la probabilidad que el modelo calculó para cada cliente.
datos1$prob_estimada <- fitted(modelo1)
```

```
ggplot(datos1, aes(x = factor(default, labels = c("Pagó", "No pagó")),
                  y = prob_estimada)) +
  geom_boxplot(fill = "grey80") +
  labs(x = "Lo que realmente pasó", y = "Probabilidad estimada de no pagar") +
  theme_minimal(base_size = 9)
```

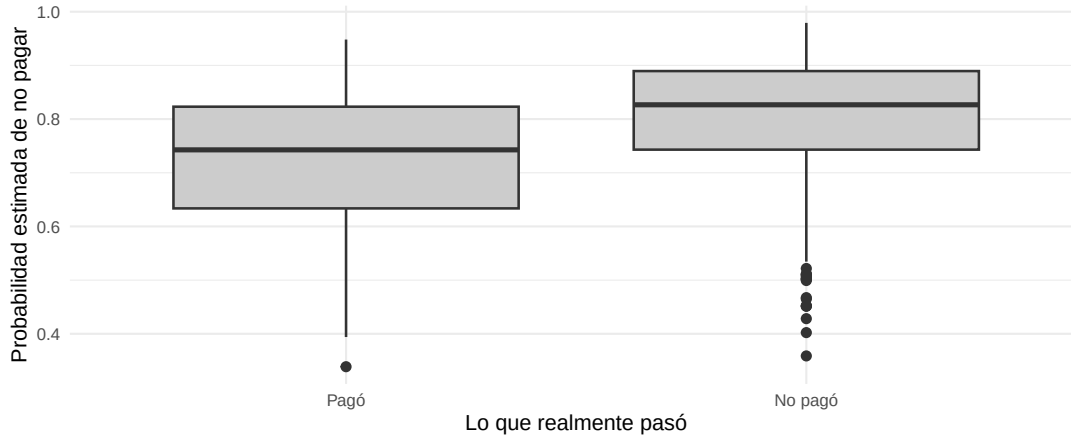


Figura 1: Parte 1. Probabilidad de no pagar que el modelo le asigna a cada cliente, separada según lo que realmente pasó. Se ve que los que no pagaron (barra de la derecha) tienden a tener probabilidades más altas, que es lo que uno espera de un modelo que funciona.

3 Parte 2: la base real Traindata

3.1 Paso 1: abrir el archivo de Excel

```
# Buscamos el Excel en los dos sitios donde suele estar: dentro de la carpeta
# "datos", o al lado de este archivo .qmd. Así funciona sin importar dónde lo
# haya guardado uno.
posibles <- c("datos/Traindata.xlsx", "Traindata.xlsx")
ruta <- posibles[file.exists(posibles)][1]

# Si no aparece en ninguno de los dos, avisamos con un mensaje claro en vez de
# dejar que R lance un error difícil de entender.
if (is.na(ruta)) {
  stop("No encuentro Traindata.xlsx. Guárdalo en la misma carpeta que este ",
       ".qmd (o dentro de una subcarpeta llamada 'datos') y vuelve a renderizar.")
}

# read_excel() abre el archivo de Excel.
traindata <- as.data.frame(read_excel(ruta))

# La columna "Score Crediticio" tiene un espacio en el nombre y eso incomoda
# a R. Le cambiamos el espacio por un guion bajo.
names(traindata)[names(traindata) == "Score Crediticio"] <- "Score_Crediticio"

# ¿Cuántas filas y columnas tiene?
dim(traindata)
```

```
#> [1] 1714    7
```

3.2 Paso 2: mirar los datos antes de modelar

```
# Las primeras filas, para saber qué pinta tienen los datos.
head(traindata, 4)
```

```
#>   Genero Moras Estado_Civil Ingresos Edad Score_Crediticio Cumple
#> 1      0      1           1  5420000  58             899      0
#> 2      0      0           1  9700000  38             785      0
#> 3      1      1           1 43080000  38             837      1
#> 4      1      1           0 38930000  39             439      1
```

```
# ¿Cuántos clientes cumplen (1) y cuántos no (0)?
table(traindata$Cumple)
```

```
#>
#>    0    1
#> 1257  457
```

La base tiene **1714 clientes** y **7 columnas**. De ellos, 457 cumplen y 1257 no cumplen, o sea que el **26.7 %** cumple.

Ojo con un detalle que cambia todo: aquí la variable a explicar se llama **Cumple**, y el 1 significa **que sí paga**. En la Parte 1 el 1 significaba lo contrario (que no paga). Entonces en esta parte **todo** se lee al revés: un OR mayor que 1 ahora es una **buena** noticia.

3.3 Paso 3: el modelo tal como lo pide la guía

La guía dice: usar **Cumple** como variable dependiente y **todas las demás** como independientes. Eso se escribe `Cumple ~ .` (el punto significa “todas las demás”). Lo hacemos:

```
modelo_completo <- glm(Cumple ~ ., family = binomial(link = "logit"),
                      data = traindata)
```

```
#> Warning: glm.fit: algorithm did not converge
```

```
#> Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(modelo_completo)
```

```
#>
#> Call:
#> glm(formula = Cumple ~ ., family = binomial(link = "logit"),
#>      data = traindata)
#>
#> Coefficients:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  -2351.076829 41395.930434  -0.06    0.95
#> Genero        8.988888   656.095310   0.01    0.99
#> Moras         0.950801   812.603451   0.00    1.00
#> Estado_Civil  23.007736   837.560057   0.03    0.98
#> Ingresos      0.000229    0.004035   0.06    0.95
#> Edad         0.793762    49.288159   0.02    0.99
#> Score_Crediticio 0.006170    2.280089   0.00    1.00
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>      Null deviance: 1987.814314239  on 1713  degrees of freedom
#> Residual deviance:   0.000011054  on 1707  degrees of freedom
#> AIC: 14
```

```
#>
#> Number of Fisher Scoring iterations: 25
```

3.4 Paso 4: algo salió mal (y hay que entender qué)

Fíjese en tres cosas de esa salida:

1. R nos dio una **advertencia**: “*algorithm did not converge*”, es decir **el modelo no pudo terminar el cálculo**.
2. Los números de la columna **Std. Error** son **gigantescos** (miles, decenas de miles). Eso quiere decir que el modelo no tiene ni idea de cuánto vale cada coeficiente.
3. **Ninguna variable salió significativa** (todos los $\Pr(>|z|)$ cerca de 1), pero al mismo tiempo la **Residual deviance** es casi 0, lo que en teoría significaría un modelo perfecto. Eso es contradictorio.

Cuando un modelo logit se comporta así, casi siempre hay una razón: **una de las variables ya contiene la respuesta**. Vamos a buscarla mirando los ingresos:

```
# Rango de ingresos de los que NO cumplen y de los que SÍ cumplen.
tapply(traindata$Ingresos, traindata$Cumple, range)
```

```
#> $`0`
#> [1] 1000000 9980000
#>
#> $`1`
#> [1] 10010000 43080000
```

```
ggplot(traindata, aes(x = factor(Cumple, labels = c("No cumple", "Cumple")),
                      y = Ingresos)) +
  geom_jitter(width = 0.25, alpha = 0.25, size = 0.6) +
  geom_hline(yintercept = 1e7, linetype = "dashed") +
  scale_y_continuous(labels = function(x) paste0(x / 1e6, " M")) +
  labs(x = NULL, y = "Ingresos (millones)") +
  theme_minimal(base_size = 9)
```

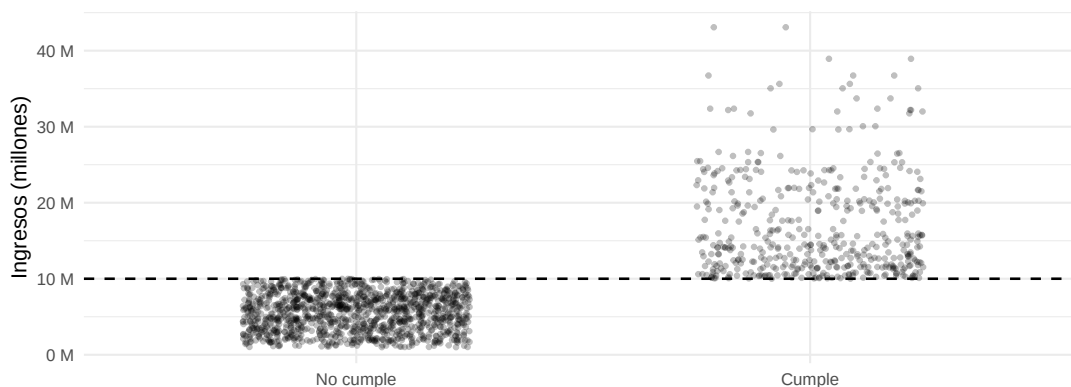


Figura 2: Parte 2. Ingresos de cada grupo. Las dos nubes de puntos no se tocan: hay una línea (10 millones) que separa perfectamente a quien cumple de quien no cumple. Esto es lo que rompió el modelo.

Aquí está el problema. Todos los que **no cumplen** ganan menos de 10 millones, y todos los que **sí cumplen** ganan más de 10 millones. Ni una sola excepción entre 1714 clientes.

Eso significa que para saber si alguien cumple **no hace falta ningún modelo**: basta con preguntar si gana más de 10 millones. Y por eso el modelo se rompe: no tiene nada que estimar, la respuesta ya venía escrita en la columna **Ingresos**. En la vida real esto nunca pasa; lo más probable es que la columna **Cumple** de este archivo se haya **construido** a partir de los ingresos con ese corte.

3.5 Paso 5: el modelo corregido

La solución es sacar **Ingresos** y quedarnos con las otras cinco variables. Así el modelo sí tiene trabajo que hacer y podemos interpretar los resultados, que es lo que pide la actividad.

```
modelo2 <- glm(Cumple ~ Genero + Moras + Estado_Civil + Edad + Score_Crediticio,
               family = binomial(link = "logit"),
               data = traindata)
```

```
summary(modelo2)
```

```
#>
#> Call:
#> glm(formula = Cumple ~ Genero + Moras + Estado_Civil + Edad +
#>       Score_Crediticio, family = binomial(link = "logit"), data = traindata)
#>
#> Coefficients:
#>              Estimate Std. Error z value      Pr(>|z|)
#> (Intercept)   -4.849268   0.425407  -11.40 <0.0000000000000002 ***
#> Genero        -0.261237   0.119895   -2.18    0.0293 *
#> Moras         1.665797   0.126994   13.12 <0.0000000000000002 ***
#> Estado_Civil  -0.228633   0.129067   -1.77    0.0765 .
#> Edad          0.023066   0.007609    3.03    0.0024 **
#> Score_Crediticio 0.003236   0.000355    9.12 <0.0000000000000002 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#> Null deviance: 1987.8  on 1713  degrees of freedom
#> Residual deviance: 1733.3  on 1708  degrees of freedom
#> AIC: 1745
#>
#> Number of Fisher Scoring iterations: 4
```

```
# Los coeficientes convertidos en multiplicadores (OR).
```

```
round(exp(coef(modelo2)), 4)
```

```
#>      (Intercept)      Genero      Moras      Estado_Civil
#>      0.0078      0.7701      5.2899      0.7956
#>      Edad Score_Crediticio
#>      1.0233      1.0032
```

3.6 Paso 6: interpretar el modelo corregido

Tabla 3: Parte 2. Resultados del modelo corregido, traducidos a lenguaje sencillo.

Si...	OR	Efecto	¿Es confiable?
Tener Moras = 1	5.290	Cumple MÁS	Sí
Score: 100 puntos más	1.382	Cumple MÁS	Sí
Edad: 10 años más	1.259	Cumple MÁS	Sí
Ser hombre (Genero = 1)	0.770	Cumple MENOS	Sí
Estado civil = 1	0.796	Cumple MENOS	No

En palabras simples (recuerde: aquí OR mayor que 1 es bueno, significa que cumple más):

- **Moras** es la variable más fuerte de todas. Con un OR de 5.29, los clientes con **Moras** = 1 tienen unas 5.3 veces más posibilidades de cumplir. **Pero cuidado:** si “Moras” significa *tener deudas atrasadas*, este resultado estaría al revés de lo lógico. Lo más probable es que en este archivo el 1 signifique *estar al día*. **Hay que confirmarlo con el profesor antes de usar este modelo para decidir algo**, porque de eso depende cómo se lee.
- **Score crediticio:** cada 100 puntos más multiplican por 1.38 las posibilidades de cumplir. Tiene todo el sentido: mejor score, mejor pagador.
- **Edad:** cada 10 años más multiplican por 1.26 las posibilidades de cumplir. Los clientes mayores pagan un poco mejor.
- **Ser hombre:** OR de 0.77, o sea que los hombres cumplen un poco menos que las mujeres.
- **Estado civil:** la columna “¿Es confiable?” dice **No** (su valor es 0.076, y para ser confiable debería ser menor que 0,05). Con estos datos no podemos afirmar que influya.

3.7 Paso 7: ¿qué tan bien funciona el modelo corregido?

```
# Regla simple: si la probabilidad es 0,5 o más, decimos que el cliente cumple.
prob2 <- fitted(modelo2)
prediccion <- ifelse(prob2 >= 0.5, 1, 0)
```

```
# Tabla de aciertos y errores.
table(Observado = traindata$Cumple, Predicho = prediccion)
```

```
#>      Predicho
#> Observado    0    1
#>      0 1139  118
#>      1  281  176
```

```
# Porcentaje de aciertos.
round(mean(prediccion == traindata$Cumple), 4)
```

```
#> [1] 0.7672
```

El modelo acierta en el **76.7 %** de los casos. Suena bien, pero hay que mirarlo con cuidado: como el 73.3 % de los clientes **no** cumple, un modelo perezoso que dijera “nadie cumple” ya acertaría el 73.3 %. Nuestro modelo mejora eso, pero poco.

Lo más flojo es que de los 457 clientes que sí cumplen, el modelo solo detecta 176 (un 38.5 %). En resumen: **el modelo sirve para ordenar clientes de más a menos riesgosos, pero no para decidir solo.**

```
resumen <- aggregate(Cumple ~ Moras, data = traindata, FUN = mean)
resumen$Moras <- factor(resumen$Moras, labels = c("Moras = 0", "Moras = 1"))

ggplot(resumen, aes(x = Moras, y = Cumple)) +
  geom_col(fill = "grey70", width = 0.5) +
  geom_text(aes(label = paste0(round(Cumple * 100, 1), " %")), vjust = -0.5,
            size = 3) +
  scale_y_continuous(limits = c(0, 0.55),
                    labels = function(x) paste0(x * 100, " %")) +
  labs(x = NULL, y = "Porcentaje que cumple") +
  theme_minimal(base_size = 9)
```

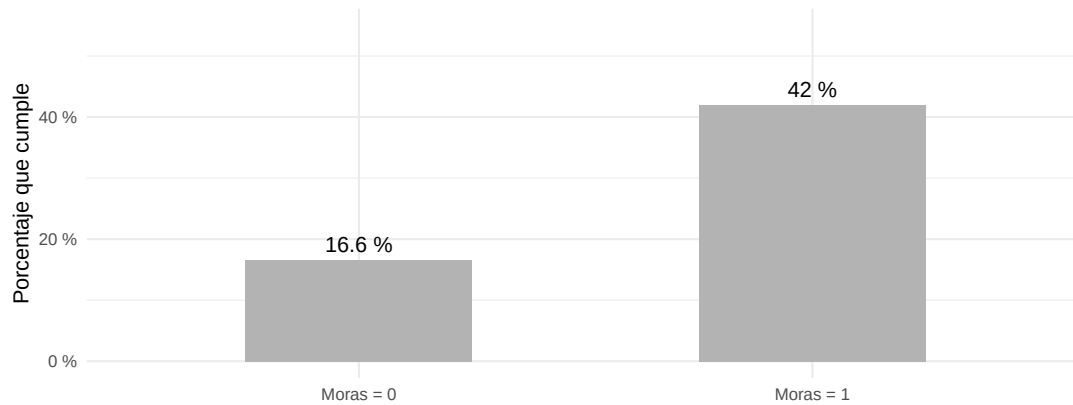


Figura 3: Parte 2. Porcentaje de clientes que cumple, según la variable Moras. Es la diferencia más grande que encontramos en toda la base, y coincide con que Moras sea la variable más fuerte del modelo.

4 Conclusiones

1. **Aprendí a construir un modelo logit en R** con la función `glm()`, tanto con datos inventados como con la base real del curso.
2. **En la Parte 1 el modelo funcionó**, pero los datos inventados tienen dos cosas poco realistas: el score sube el riesgo (debería bajarlo) y el 78.5 % de incumplimiento es demasiado alto. Eso viene de la fórmula que trae la guía, no de un error de programación.
3. **En la Parte 2 el modelo con todas las variables falló**, y encontré la razón: la columna **Ingresos** separa perfectamente a quien cumple de quien no (corte en 10 millones). Al quitarla, el modelo funcionó normal.
4. **En el modelo corregido, las variables que más importan son Moras y el Score_Crediticio**. El estado civil no resultó confiable.
5. **Lo más importante que me deja el taller:** antes de confiar en un modelo hay que mirar los datos y revisar que los resultados tengan sentido. Los dos problemas que encontré (el score al revés y los ingresos que lo delataban todo) no los detecta ningún programa solo: hay que revisarlos uno mismo.
6. **Queda una pregunta pendiente para el profesor:** ¿qué significa el 1 en la columna **Moras**? Según la respuesta, la interpretación de la variable más importante del modelo cambia por completo.

5 Referencias

- Contenido del Módulo 1, *Fundamentos de Inteligencia Artificial y Automatización de Procesos Financieros*.
- Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media, capítulos 1 y 2.
- Base de datos **Traindata.xlsx**, aula virtual del curso, Módulo 1.
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*.